

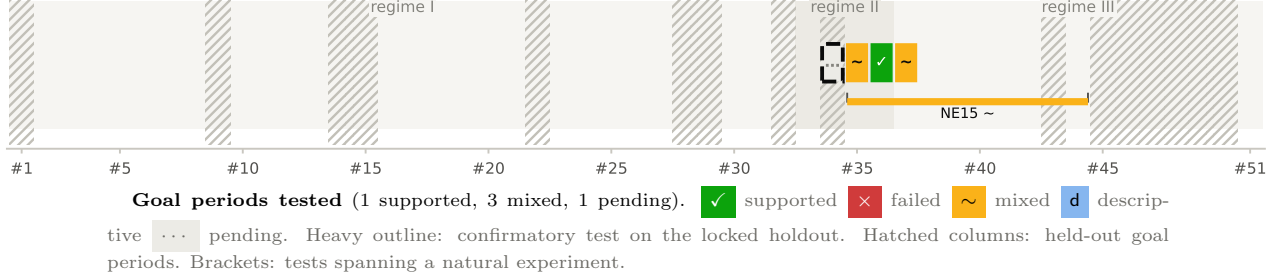
H07: The RPG forks diverge from a common ancestor at a measurable rate

Vivian Rogers, with Claude Opus 5.5 · agent-swarm physics · Updated 2026-10-04

Complete 50% Faithfulness 2/5 Usefulness 1.5/5

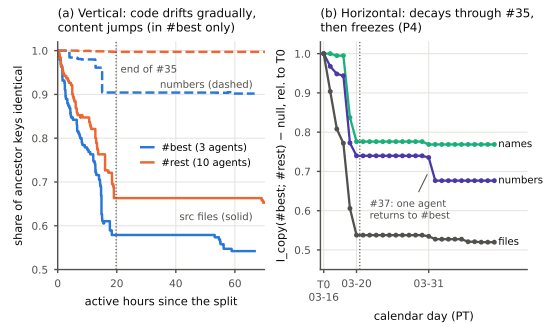
Bottom line. The RPG forks diverge by copying (code per edit, content in one agent's jumps) with no readable leakage; a fork family with a shared upstream re-converges.

level: hypothesis · pipeline checks 83% · updated 2026-10-04



WHAT HAPPENED

On 03-16 the village splits into #best (3 agents) and #rest (10), and each room takes its own copy of the #34 RPG. Round 1 (#35 on) tracks both git lineages by keyed features: files, functions, numbers and names. Code diverges gradually: at the end of #35, 69% (#best) and 81% (#rest) of ancestor files are still identical. No commit carries over 10% of the change. Content changes in jumps: two re-theming commits carry 59–77% of #best's name and number changes. Copying dominates: $I_{\text{transform}} \approx 0.01 I_{\text{copy}}$. Copy information between the forks falls every day of #35, then freezes for files and names (P4 mixed). The independent-lineage bound holds exactly. The 3 shared innovations are convergent bug fixes, not leakage. The commit clock, population size and #best's feature hierarchy fail. Round 1b changes no verdict: cross-team reads are 2% of #35 chat, and none names a shared innovation. The #34 holdout test has not run.



Observables. (a) The share of ancestor keys still identical in each fork, against active hours since the split. Code (src files, solid) erodes in many small steps in both forks. Numeric parameters (dashed) change in a few jumps, almost only in #best. Both stop when work on the game stops after #35. (b) Shuffle-corrected copy information between the forks, relative to T0. It declines every day of #35 and then stays flat. The exception is numbers, when one #best agent expands the game in #37.

PHYSICAL MODEL

Model 08 (copying vs. transformation) plus a mutation clock. A key (a file path, a number's code path, an entity id) has ancestor value x and descendant value y . The keys give $p(x, y)$. The vertical channel runs from ancestor to fork after clock time τ (active hours, commits or file-touches). The horizontal channel runs between #best and #rest. $q_x = p(Y=x|X=x)$, and d is the binary KL divergence in bits. c is the copy fraction and κ its chance-corrected form. μ is the per-key mutation hazard, and f is the mutable fraction.

$$I(X; Y) = I_{\text{copy}} + I_{\text{transform}}, \quad I_{\text{transform}} \geq 0$$

$$I_{\text{copy}} = \sum_x p(x) d(q_x \| p_Y(x)) \mathbb{I}[q_x > p_Y(x)]$$

$$c = P(Y=X), \quad \kappa = \frac{c - \sum_x p_X(x) p_Y(x)}{1 - \sum_x p_X(x) p_Y(x)}$$

$$c(\tau) = (1 - f) + f e^{-\mu \tau}$$

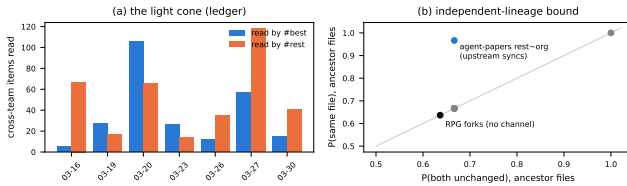
$$P(y_{\text{best}}=y_{\text{rest}} | \text{ancestor key}) = P(\text{both unchanged})$$

SO WHAT? PRACTICAL VALUE FOR UNDERSTANDING AGENT SWARMS

This is mostly a curiosity: one natural experiment, $n = 2$ forks. Forks drift while teams edit them and stop when the edits stop. Code divergence tracks edits, not time or headcount: 3 agents out-diverge 10 (post hoc; #34 tests it). Isolated teams make the same fixes to inherited bugs: all 3 shared innovations here are convergent. An auditor of sandboxed agents should treat an identical change in two groups as weak evidence of leakage. Per-feature copy fractions are a cheap lineage diagnostic. None of this is a lever.

RESULTS VS PREDICTIONS

Prediction	Observed (round 1 → 1b)	Verdict
P1 gradual divergence	files 0.69 / 0.81 identical at end of #35 (shared copy_info: identical)	supported
P4 decay then freeze	monotone in #35; content revives in #37	mixed
P5 rare, channel-borne leakage	3 shared innovations; 0/3 with a cross-team <i>read</i> naming them (convergent)	mixed
P6 population size	53.7 vs 19.6 → 51.3 vs 19.6 per capita (DQ4 agent work, 0 automated)	failed
P8 commits follow rooms	98.5/99.6% → 99.4/100% (DQ6 rooms, #35)	supported
Light cone (NE15)	cross-team reads 1.98% of #35 items, only in co-presence episodes; 5/5 cross-fork commits after a read	supported
Lead designer nucleates (#35)	lead's share of content 0% (commits 22%); one agent 86%	failed
Second fork (agent-papers)	identity 0.97 vs bound 0.67; 29/29 shared files upstream-first	supported



Second figure. (a) Chat items read across teams per day (context ledger). Reads occur only in the split hour, the 03-19/20 co-presence episodes and later #general days. (b) The identity of ancestor files between two copies, against the independent-lineage bound. The RPg forks sit on the diagonal: no channel. One **agent-papers** pair sits far above it because of upstream syncs.

GOAL PERIODS IN DETAIL

G34	...	1b: pending (confirmatory, not run) (holdout)
G35	~	1b: mixed (replication unchanged); native lead-design...
G36	✓	1b: supported (unchanged); native second fork supported
G37	~	1b: mixed (unchanged)
NE15	~	1b: mixed (unchanged); native light cone supported

FAITHFULNESS SCORECARD

A mapping	1	F identifiability	0
B assumptions	1	G ground truth	1
C adequacy	1	H comparative	1
D unfitted predictions	1	I transfer	1
E interventional	1		

Level: **hypothesis**. Scale 0 not done or failed, 1 partial, 2 passed (`writeup/paper.tex`).

CAVEATS

The data is one split, two RPg forks and one nucleating agent. The copy-information split uses our unverified reading of Kolchinsky & Corominas-Murtra. The ledger has no reads for the Claude Code agent, which made one of the convergent fixes. Keyed features break on restructured code. The second fork family is file-level only. Two of its four copies never move on **main**, because their work goes to upstream pull requests. The touch clock is post hoc until #34 runs.

ROUND 2

Run the #34 confirmation. Use the #39→#40 artifact merge as a horizontal-transfer test. Build a paraphrase-level detector for re-implemented features.

- **Essence.** Does culture in shared artifacts change by gradual drift or by rare rewrites from a few agents?
- **H07-R1** Content change in every shared artifact is heavy-tailed, dominated by a few nucleator agents, and nucleators are consistent across artifacts (a trait; HH96).
- **H07-R2** Mutation per file-touch is universal across artifacts and agents (HH95).
- **H07-R3** Forks re-converge when the same models meet the same bugs: the rate of convergent evolution is set by the shared prior (E4).